

FACULTY OF COMPUTERS AND INFORMATION

Department of Computer Science

SOFTWARE REQUIREMENTS
SPECIFICATION

IEEE Std 830-1998 Compliant

Hotel Management System

Full System Documentation with UML Diagrams & Design Patterns

Course	Software Engineering
Academic Year	2025 / 2026
Semester	Second Semester
Submission Date	May 2026
Document Version	Version 1.0 — Final

PROJECT TEAM MEMBERS

#	Student ID	Student Name
1	20230159	جني نادر ناصف
2	20230176	حسن محمد حسن ابو ريه
3	20230290	شيماء احمد محمد
4	20230291	شيماء فكري عزت
5	20230572	ملك حسن عبدالمنعم هاشم
6	20230590	منة الله هشام عثمان

SECTION 1 — INTRODUCTION

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the Hotel Management System (HMS). It is intended to guide the development team, project managers, and stakeholders throughout the software development lifecycle.

1.2 Scope

The Hotel Management System is a full-stack web application designed to automate hotel operations. It supports three user roles — Customer, Receptionist, and Admin — and provides capabilities including room browsing, booking management, payment processing, check-in/check-out operations, reviews, wishlists, and notifications.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
SRS	Software Requirements Specification
HMS	Hotel Management System
JWT	JSON Web Token — used for authentication
RBAC	Role-Based Access Control
OTP	One-Time Password — used for email verification
API	Application Programming Interface
UC	Use Case
FR	Functional Requirement
NFR	Non-Functional Requirement
CRUD	Create, Read, Update, Delete
UI	User Interface
DB	Database
UML	Unified Modeling Language

1.4 References

- Hotel Management System Scenarios PDF — provided by project team
- SW Projects documentation — entity and module definitions
- UML Hotel diagrams — attached use-case and sequence diagrams
- IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications

1.5 Overview

The remainder of this SRS is organized as follows: Section 2 provides an overall description; Section 3 details functional requirements with use cases; Section 4 covers non-functional requirements; Section 5 presents the data model; Section 6 describes the system architecture; Sections 7–12 contain UML diagrams including use-case, class, sequence, activity, and ER diagrams.

SECTION 2 — OVERALL DESCRIPTION

2. Overall System Description

2.1 System Perspective

The Hotel Management System is a standalone, web-based application that integrates with external payment gateways (Stripe / PayPal) and notification services (Email / SMS). It follows a microservices architecture using Spring Boot for the backend and a modern frontend framework for the client interface.

2.2 System Functions (Summary)

Core System Functions
➤ User Registration, Login, and Role-Based Access (Customer / Receptionist / Admin)
➤ Room Browsing with Filters (type, price range, availability, amenities)
➤ Booking / Reservation Management (create, modify, cancel, view history)
➤ Secure Payment Processing (Online via gateway or Cash at hotel)
➤ Check-In and Check-Out Operations managed by Receptionist
➤ Review & Rating System for completed stays
➤ Wishlist / Favorites management per customer
➤ Automated Notification System (Email / SMS / OTP)
➤ Admin Dashboard with analytics, user management, and system monitoring
➤ Receptionist Dashboard for daily hotel operations

2.3 User Classes and Characteristics

Role	Technical Level	Primary Responsibilities
Customer	Basic user	Browse rooms, make bookings, pay, review stays, manage wishlist
Receptionist	Intermediate	Handle check-in/check-out, manage bookings, process cash payments
Admin	Advanced	Full system control: manage users, rooms, view analytics, monitor logs

2.4 Operating Environment

- Backend: Spring Boot microservices — Java 17+
- Frontend: React.js / Angular — runs in modern web browsers
- Database: MySQL / PostgreSQL
- Authentication: JWT tokens with refresh mechanism
- Hosting: Cloud-based (AWS / Azure / GCP)
- Browsers: Chrome 90+, Firefox 88+, Safari 14+, Edge 90+

2.5 Design and Implementation Constraints

- All passwords must be encrypted using bcrypt before storage
- JWT tokens expire after 24 hours; refresh tokens valid for 7 days
- The system must support at least 500 concurrent users
- All API endpoints must be secured and role-restricted
- Payment integration must comply with PCI-DSS standards
- The system must log all critical actions for audit purposes (AOP)

2.6 Assumptions and Dependencies

- Customers must have a valid email address for account registration
- The hotel has a fixed set of room types (Single, Double, Suite)
- Internet connectivity is required for online payment processing
- Third-party SMS/Email services (e.g., Twilio, SendGrid) are available

SECTION 3 — SYSTEM ACTORS

3. System Actors

3.1 Actor Overview Diagram



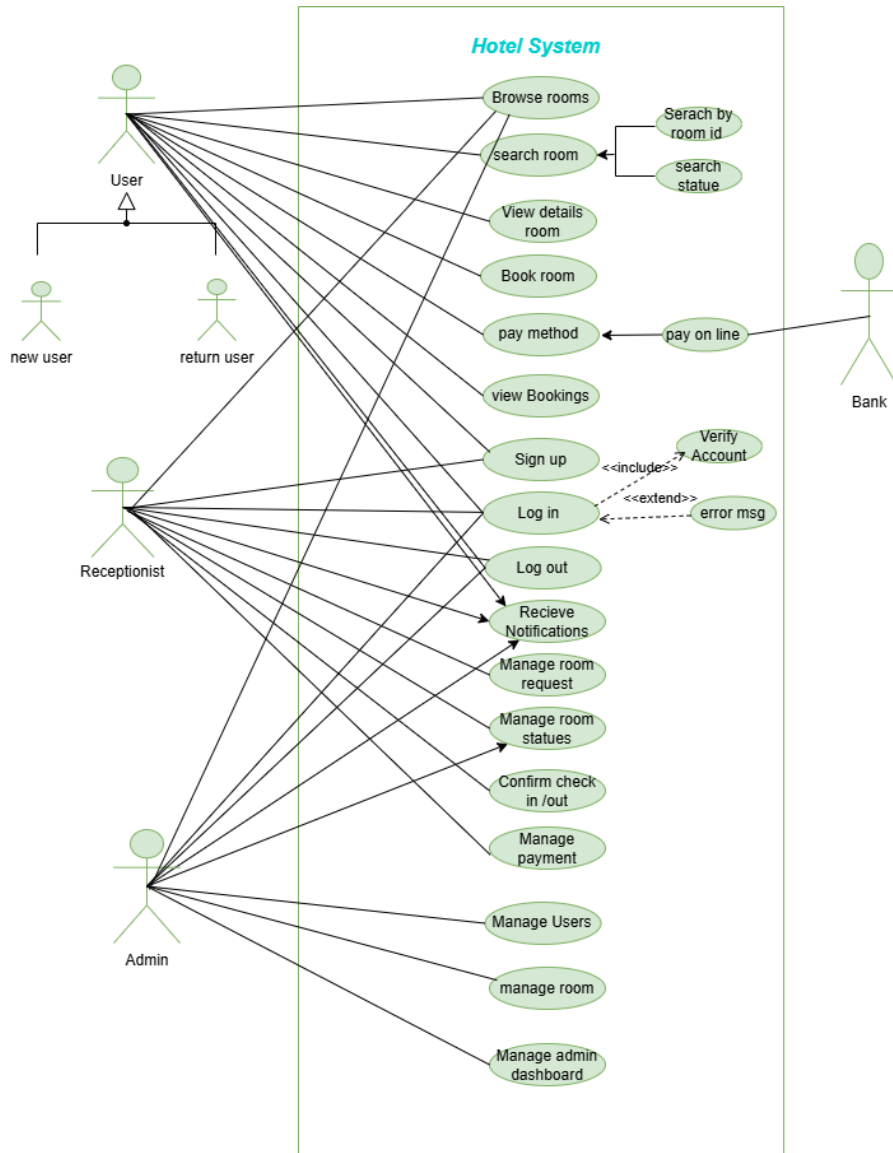
3.2 Actor Descriptions

Actor	Description
Customer	End-user who registers an account, browses available rooms, makes reservations, processes payments, manages a wishlist, and submits reviews after completing a stay.
Receptionist	Hotel staff member who manages daily operations: viewing and confirming bookings, performing check-in and check-out, updating room statuses, handling cash payments, and generating invoices.
Admin	System administrator with full access to all modules. Responsible for user management, room management, system configuration, monitoring analytics, and reviewing audit logs.
Payment Gateway	External system (Stripe / PayPal) that processes secure online credit/debit card transactions.
Email/SMS Service	External notification service (SendGrid / Twilio) that sends booking confirmations, OTP codes, reminders, and receipts to users.

SECTION 4 — USE CASES & SCENARIOS

4. Use Cases and Scenarios

4.1 Use Case Diagram — Full System



4.2 Use Case: UC-01 — User Registration

Use Case ID	UC-01
Use Case Name	User Registration
Actor(s)	Customer (new user)
Pre-condition	User does not have an existing account; application is accessible

Main Flow	1. User opens the registration page 2. User enters: name, email, phone number, and password 3. System validates all fields (format, uniqueness of email) 4. System sends OTP to the provided email 5. User enters OTP 6. System verifies OTP and creates account with role = CUSTOMER 7. System redirects user to the Customer Dashboard
Alternative Flow	A1: Email already exists → system shows error "Email already registered" A2: Invalid OTP → system asks user to resend OTP A3: Weak password → system shows password strength requirements
Post-condition	New user account created and stored in database; user is authenticated
Business Rules	Password must be at least 8 characters with mixed case and numbers; email must be unique

4.3 Use Case: UC-02 — User Login

Use Case ID	UC-02
Use Case Name	User Login
Actor(s)	Customer, Receptionist, Admin
Pre-condition	User must have a registered account
Main Flow	1. User opens the login page 2. User enters email and password 3. System validates credentials against the database 4. System generates a JWT token and assigns role-based session 5. System redirects user to their role-specific dashboard
Alternative Flow	A1: Incorrect credentials → system shows "Invalid email or password" (max 5 attempts) A2: Account blacklisted → system shows "Account suspended" A3: Forgot password → user clicks link, receives email OTP to reset
Post-condition	User is authenticated; JWT stored in client; session active
Business Rules	After 5 failed attempts, account is temporarily locked for 15 minutes

4.4 Use Case: UC-03 — Browse and Filter Rooms

Use Case ID	UC-03
Use Case Name	Browse and Filter Rooms
Actor(s)	Customer
Pre-condition	User may be logged in or browsing as guest
Main Flow	1. User accesses the Home page / Rooms page 2. System displays all available rooms with photos, price, type, and features 3. User applies filters: room type, price range, check-in/check-out dates, amenities 4. System queries database and returns matching rooms 5. User selects a room to view full details

Alternative Flow	A1: No rooms match filters → system shows "No rooms available for selected criteria" A2: User resets filters → all rooms displayed again
Post-condition	User sees filtered list of rooms matching criteria
Business Rules	Unavailable rooms (Booked / Maintenance) must be clearly marked

4.5 Use Case: UC-04 — Make Reservation

Use Case ID	UC-04
Use Case Name	Make Reservation / Booking
Actor(s)	Customer
Pre-condition	Customer is logged in; room is available for selected dates
Main Flow	1. Customer selects a room and clicks "Book Now" 2. Customer enters check-in date and check-out date 3. System checks room availability for selected period 4. System calculates total price = price_per_night × number_of_nights 5. System displays booking summary (room details, total cost, dates) 6. Customer confirms booking 7. System creates Booking record with status = "Pending" 8. System sends booking confirmation notification to customer
Alternative Flow	A1: Room is unavailable for selected dates → system shows conflict message A2: Customer cancels → no booking created
Post-condition	Booking record created with status "Pending"; notification sent
Business Rules	Minimum stay: 1 night; Maximum advance booking: 365 days

4.6 Use Case: UC-05 — Payment Process

Use Case ID	UC-05
Use Case Name	Payment Process
Actor(s)	Customer, Payment Gateway (external)
Pre-condition	Booking record exists with status "Pending"
Main Flow	1. Customer proceeds to payment after booking confirmation 2. System presents payment options: Online or Cash 3a. (Online) Customer enters card details; system connects to Payment Gateway 3b. Payment Gateway processes transaction and returns success response 4. System updates Payment status = "Paid" and Booking status = "Confirmed" 5. System generates invoice and sends payment confirmation notification 5b. (Cash) Booking remains "Pending Payment" until paid at hotel
Alternative Flow	A1: Online payment fails → system shows error; booking remains Pending A2: Customer selects Cash → receptionist marks as paid during check-in
Post-condition	Payment recorded; invoice generated; booking status updated
Business Rules	Online payments use encrypted HTTPS channels; PCI-DSS compliant

4.7 Use Case: UC-06 — Check-In

Use Case ID	UC-06
Use Case Name	Check-In Process
Actor(s)	Receptionist
Pre-condition	Booking exists with status "Confirmed"; customer is present at hotel
Main Flow	1. Receptionist searches for booking by Booking ID or customer name 2. System retrieves and displays booking details 3. Receptionist verifies customer identity and booking validity 4. Receptionist performs check-in action 5. System assigns room to customer 6. System updates Room status from "Available" to "Booked" 7. System updates Booking status to "Active" / "Checked-In" 8. System records check-in timestamp
Alternative Flow	A1: Booking not found → system shows error A2: Payment not completed → receptionist collects cash before check-in
Post-condition	Room marked as Booked; check-in recorded; customer notified
Business Rules	Standard check-in time: 14:00; early check-in subject to availability

4.8 Use Case: UC-07 — Check-Out

Use Case ID	UC-07
Use Case Name	Check-Out Process
Actor(s)	Receptionist
Pre-condition	Booking is in "Checked-In" status; customer is ready to leave
Main Flow	1. Receptionist initiates check-out for customer 2. System calculates final bill including room charges and any extra services 3. Receptionist verifies payment completion 4. System updates Booking status to "Completed" 5. System updates Room status to "Available" or "Maintenance" 6. System generates final invoice 7. System sends checkout notification and invoice to customer 8. System enables customer to submit a review
Alternative Flow	A1: Outstanding payment → receptionist collects before completing check-out A2: Late check-out → additional charges applied per hotel policy
Post-condition	Booking marked Completed; room freed; invoice sent; review enabled
Business Rules	Standard check-out time: 12:00; late check-out fee applies after 14:00

4.9 Use Case: UC-08 — Submit Review

Use Case ID	UC-08
Use Case Name	Submit Review and Rating

Actor(s)	Customer
Pre-condition	Customer has completed a stay (Booking status = "Completed")
Main Flow	1. Customer navigates to the Reviews section 2. System lists all completed stays eligible for review 3. Customer selects a booking and enters: rating (1–5 stars) and comment 4. System validates and stores the review linked to customer and room 5. System displays updated average room rating
Alternative Flow	A1: Customer has already reviewed this stay → system shows existing review A2: Admin flags review as inappropriate → review hidden pending moderation
Post-condition	Review stored and linked to room; room average rating updated
Business Rules	One review per completed stay; minimum 10-character comment required

4.10 Use Case: UC-09 — Manage Wishlist

Use Case ID	UC-09
Use Case Name	Wishlist Management
Actor(s)	Customer
Pre-condition	Customer is logged in
Main Flow	1. Customer browses rooms 2. Customer clicks "Add to Wishlist" on any room 3. System creates a Wishlist record linking user_id and room_id 4. Customer can navigate to "My Favorites" to view saved rooms 5. Customer can remove rooms from wishlist
Alternative Flow	A1: Room already in wishlist → system shows "Already in favorites"
Post-condition	Wishlist record updated in database
Business Rules	Maximum 50 rooms per wishlist per user

4.11 Use Case: UC-10 — Admin — Manage Users

Use Case ID	UC-10
Use Case Name	Admin — Manage Users
Actor(s)	Admin
Pre-condition	Admin is logged in with ADMIN role
Main Flow	1. Admin navigates to User Management page 2. System lists all registered users with their roles and status 3. Admin can: view user profiles, change roles, flag as VIP, blacklist users 4. Admin confirms action 5. System updates user record and sends notification if required
Alternative Flow	A1: Admin tries to delete the only Admin account → system prevents this
Post-condition	User record updated; action logged in audit trail

Business Rules	Only Admin can change user roles; all actions are logged via AOP
-----------------------	--

SECTION 5 — FUNCTIONAL REQUIREMENTS

5. Functional Requirements

5.1 Authentication & Authorization Module (FR-AUTH)

FR ID	Requirement
FR-AUTH-01	The system shall allow new users to register with name, email, phone, and password.
FR-AUTH-02	The system shall validate the uniqueness of email during registration.
FR-AUTH-03	The system shall send an OTP to the user email for account verification.
FR-AUTH-04	The system shall authenticate users via email and password and issue JWT tokens.
FR-AUTH-05	The system shall implement Role-Based Access Control (RBAC) with roles: CUSTOMER, RECEPTIONIST, ADMIN.
FR-AUTH-06	The system shall encrypt all passwords using bcrypt before storage.
FR-AUTH-07	The system shall lock accounts after 5 consecutive failed login attempts.
FR-AUTH-08	The system shall provide a password reset flow using OTP/email.
FR-AUTH-09	The system shall support user classification: VIP and Blacklisted.
FR-AUTH-10	The system shall allow users to update their profile information.

5.2 Room Management Module (FR-ROOM)

FR ID	Requirement
FR-ROOM-01	The system shall allow Admin to add, edit, and delete room records.
FR-ROOM-02	The system shall support room types: Single, Double, Suite.
FR-ROOM-03	The system shall maintain room statuses: Available, Booked, Maintenance.
FR-ROOM-04	The system shall store room features: WiFi, AC, ocean view, king bed, balcony, mini bar, smart TV.
FR-ROOM-05	The system shall allow rooms to have multiple image uploads.
FR-ROOM-06	The system shall display all available rooms to customers on the home page.
FR-ROOM-07	The system shall support filtering rooms by type, price range, availability, and features.

FR-ROOM-08	The system shall display room details including price per night, description, and max capacity.
-------------------	---

5.3 Booking Module (FR-BOOK)

FR ID	Requirement
FR-BOOK-01	The system shall allow logged-in customers to create a booking.
FR-BOOK-02	The system shall check room availability for the selected date range before confirming.
FR-BOOK-03	The system shall calculate $\text{total_price} = \text{price_per_night} \times \text{number_of_nights}$.
FR-BOOK-04	The system shall support booking statuses: Pending, Confirmed, Cancelled, Completed.
FR-BOOK-05	The system shall allow customers to view their full booking history.
FR-BOOK-06	The system shall allow customers to cancel a booking before check-in.
FR-BOOK-07	The system shall allow receptionists to accept or reject booking requests.
FR-BOOK-08	The system shall send a notification upon booking creation and confirmation.

5.4 Payment Module (FR-PAY)

FR ID	Requirement
FR-PAY-01	The system shall support two payment methods: Online (via gateway) and Cash.
FR-PAY-02	The system shall integrate with a payment gateway (Stripe / PayPal) for online payments.
FR-PAY-03	The system shall maintain payment statuses: Paid and Pending.
FR-PAY-04	The system shall generate an invoice upon successful payment.
FR-PAY-05	The system shall send a payment confirmation notification to the customer.
FR-PAY-06	The system shall allow receptionists to record and manage cash payments.
FR-PAY-07	The system shall store payment records linked to booking records.

SECTION 6 — NON-FUNCTIONAL REQUIREMENTS

6. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	The system shall respond to API requests within 2 seconds under normal load (up to 500 concurrent users).
NFR-02	Performance	Database queries shall execute within 500ms for standard read operations.
NFR-03	Scalability	The microservices architecture shall support horizontal scaling of individual services.
NFR-04	Availability	The system shall maintain 99.5% uptime (approximately 43 hours downtime per year maximum).
NFR-05	Security	All passwords shall be hashed using bcrypt with a minimum work factor of 12.
NFR-06	Security	All API endpoints shall require valid JWT tokens (except registration and login).
NFR-07	Security	All data transmission shall use HTTPS/TLS 1.2 or higher.
NFR-08	Security	Payment processing shall comply with PCI-DSS Level 1 standards.
NFR-09	Usability	The UI shall be responsive and support mobile, tablet, and desktop screen sizes.
NFR-10	Usability	The system shall display appropriate error messages for all invalid operations.
NFR-11	Maintainability	All critical system actions shall be logged using AOP (Aspect-Oriented Programming).
NFR-12	Maintainability	Code shall follow SOLID principles; each service shall have a single responsibility.
NFR-13	Reliability	The system shall handle payment gateway failures gracefully and retry up to 3 times.
NFR-14	Reliability	The system shall implement database transaction rollback on partial failures.
NFR-15	Portability	The system shall be containerized using Docker for consistent deployment.

SECTION 7 — DATA MODEL & ENTITIES

7. Data Model

7.1 Entity: User

Field Name	Data Type	Description
id	UUID / Long	Primary key, auto-generated
name	VARCHAR(100)	Full name of the user
email	VARCHAR(150)	Unique email address (used for login)
password	VARCHAR(255)	Bcrypt-hashed password
phone	VARCHAR(20)	Contact phone number
role	ENUM	ADMIN / RECEPTIONIST / CUSTOMER
is_vip	BOOLEAN	Marks user as VIP customer (default: false)
is_blacklisted	BOOLEAN	Marks user as blocked (default: false)
created_at	TIMESTAMP	Account creation timestamp

7.2 Entity: Room

Field Name	Data Type	Description
id	UUID / Long	Primary key
room_number	VARCHAR(10)	Unique room identifier (e.g., "101")
type	ENUM	Single / Double / Suite
status	ENUM	Available / Booked / Maintenance
price_per_night	DECIMAL(10,2)	Nightly rate in local currency
description	TEXT	Room description and overview
max_users	INT	Maximum occupancy capacity
imgs	JSON / Array	List of image URLs
ocean_view	BOOLEAN	Has ocean / sea view
king_bed	BOOLEAN	Has king-size bed

balcony	BOOLEAN	Has balcony
mini_bar	BOOLEAN	Has mini bar
wifi	BOOLEAN	WiFi included
smart_tv	BOOLEAN	Smart TV available

7.3 Entity: Booking

Field Name	Data Type	Description
id	UUID / Long	Primary key
user_id	FK → User	Reference to the customer who made the booking
room_id	FK → Room	Reference to the booked room
check_in_date	DATE	Scheduled check-in date
check_out_date	DATE	Scheduled check-out date
status	ENUM	Pending / Confirmed / Cancelled / Completed
total_price	DECIMAL(10,2)	Total cost for the entire stay
phone	VARCHAR(20)	Contact phone for this booking
created_at	TIMESTAMP	Booking creation timestamp

7.4 Entity: Payment

Field Name	Data Type	Description
id	UUID / Long	Primary key
booking_id	FK → Booking	Reference to the associated booking
amount	DECIMAL(10,2)	Amount paid or due
method	ENUM	Cash / Online
status	ENUM	Paid / Pending
payment_date	TIMESTAMP	Date and time of successful payment

7.5 Entity: Review

Field Name	Data Type	Description
------------	-----------	-------------

id	UUID / Long	Primary key
user_id	FK → User	Reference to the reviewing customer
room_id	FK → Room	Reference to the reviewed room
rating	INT (1–5)	Star rating from 1 (worst) to 5 (best)
comment	TEXT	Written feedback from the customer
created_at	TIMESTAMP	Review submission timestamp

7.6 Entity: Notification

Field Name	Data Type	Description
id	UUID / Long	Primary key
user_id	FK → User	Reference to the recipient user
message	TEXT	Notification content
type	ENUM	Email / SMS
status	ENUM	Sent / Pending
created_at	TIMESTAMP	Notification creation timestamp

7.7 Entity: WishList

Field Name	Data Type	Description
id	UUID / Long	Primary key
user_id	FK → User	Reference to the customer
room_id	FK → Room	Reference to the saved room
added_at	TIMESTAMP	Timestamp when room was added to wishlist

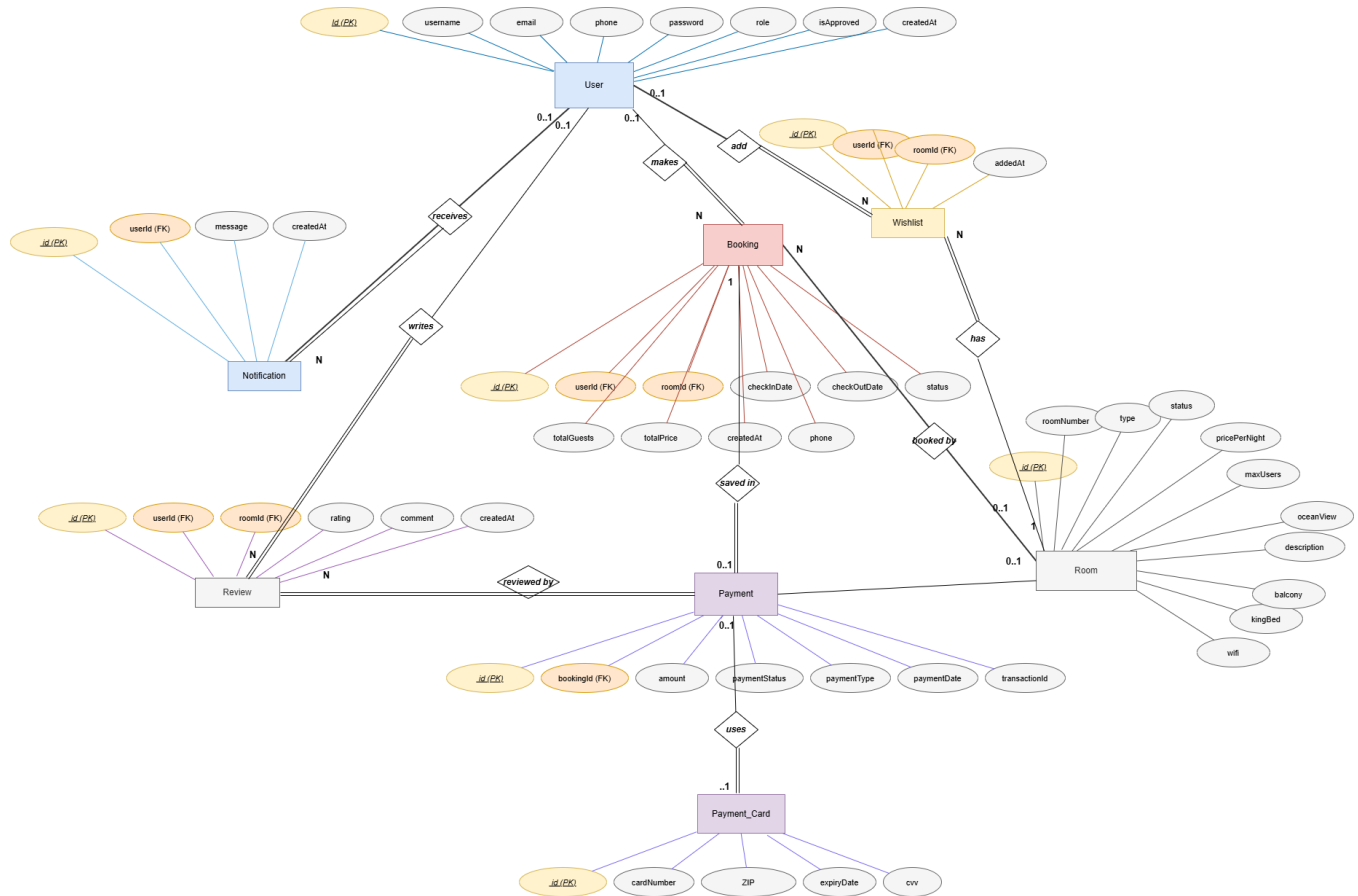
SECTION 8 — ENTITY RELATIONSHIPS

8. Entity Relationships

8.1 Relationship Table

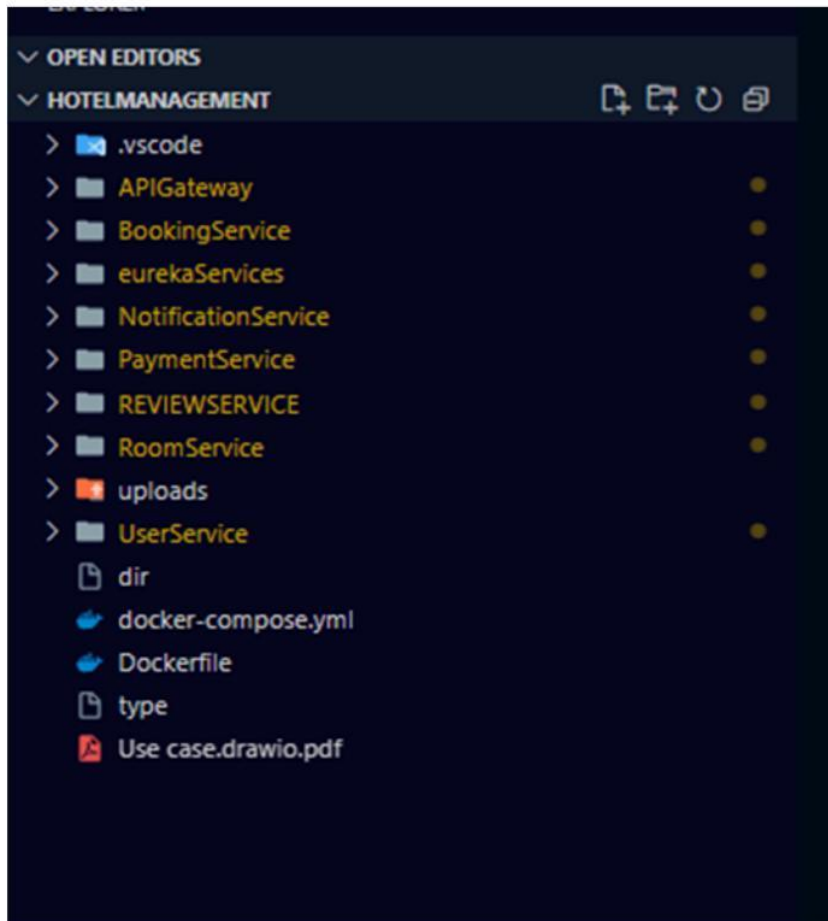
Entity A	Relationship	Entity B	Description
User	One-to-Many (1:N)	Booking	One user can make many bookings; each booking belongs to one user
Room	One-to-Many (1:N)	Booking	One room can have many bookings (across different time periods)
Booking	One-to-One (1:1)	Payment	Each booking has exactly one payment record
User	One-to-Many (1:N)	Review	One user can submit many reviews (one per completed stay)
Room	One-to-Many (1:N)	Review	One room can receive many reviews from different customers
User	One-to-Many (1:N)	Notification	One user can receive many notifications
User	One-to-Many (1:N)	WishList	One user can have many wishlist entries
Room	One-to-Many (1:N)	WishList	One room can appear in many users' wishlists

8.2 Entity-Relationship (ER) Diagram



9.3 Microservices Breakdown

Service	Responsibilities
User Service	Registration, login, JWT issuance, RBAC, profile management, VIP/blacklist classification, OTP/email verification
Room Service	Room CRUD operations, availability tracking, filter/search, amenities, wishlist management
Booking Service	Reservation creation, availability checks, booking status management, check-in/check-out coordination, favorites module
Payment Service	Online payment gateway integration (Stripe/PayPal), invoice generation, cash payment tracking, payment status updates
Notification Service	Email/SMS dispatch, OTP generation, booking confirmations, reminders, AOP-triggered alerts
Review Service	Review/rating CRUD, moderation support, average rating computation

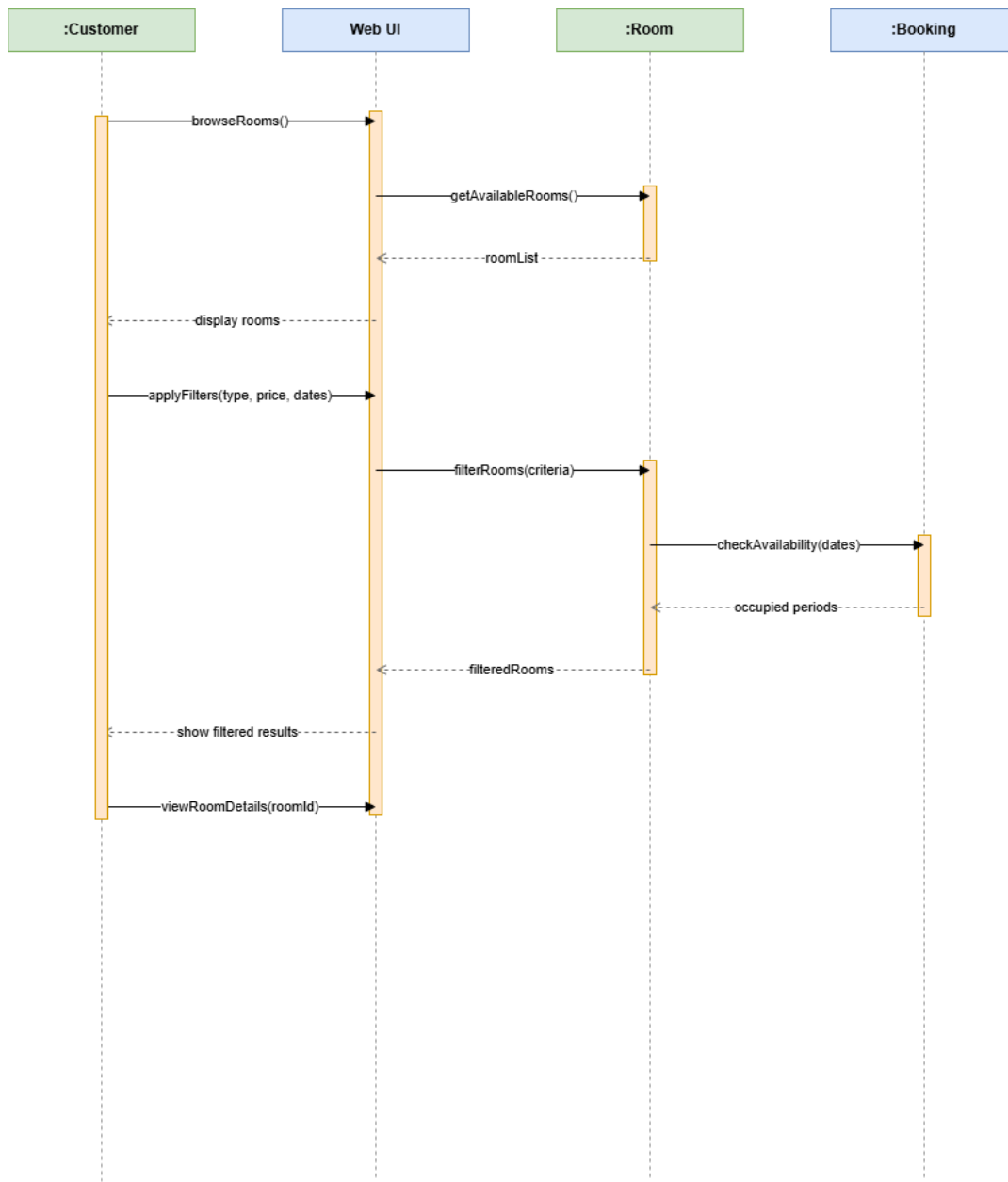


SECTION 10 — SEQUENCE DIAGRAMS

10. Sequence Diagrams

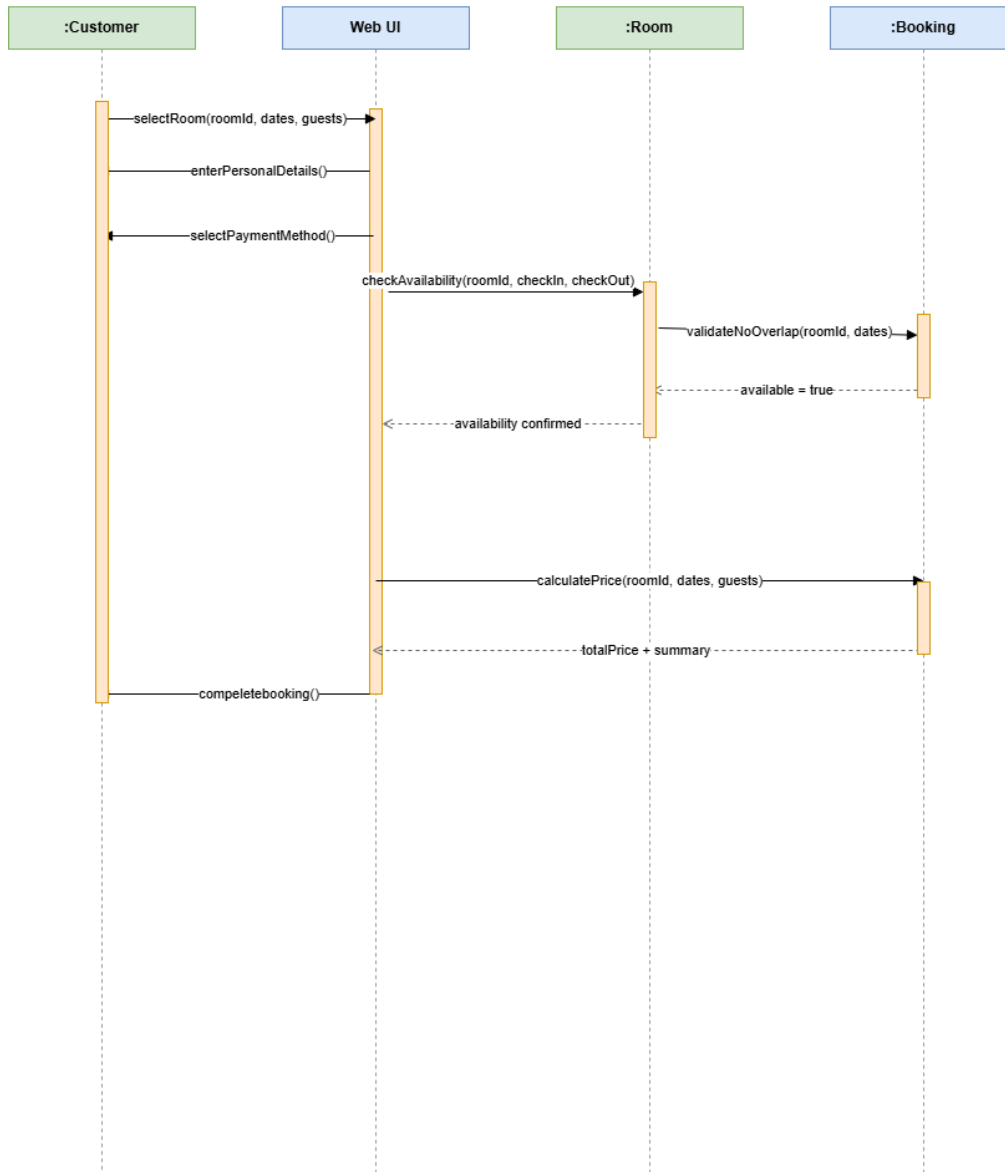
10.1 Sequence Diagram — BrowseRoom

Sequence - Browse Rooms

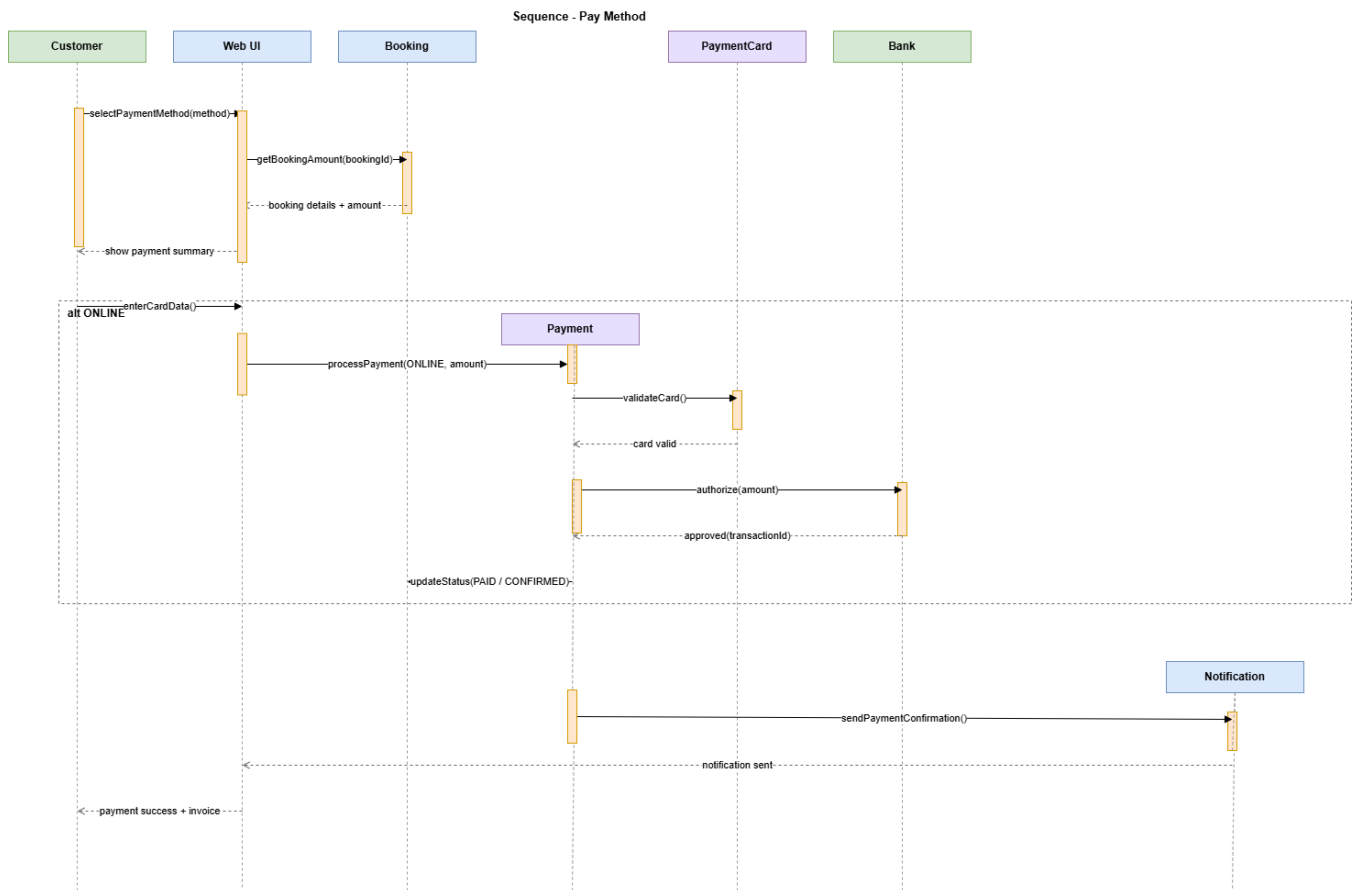


10.2 Sequence Diagram — Make Booking

Sequence - Book Room

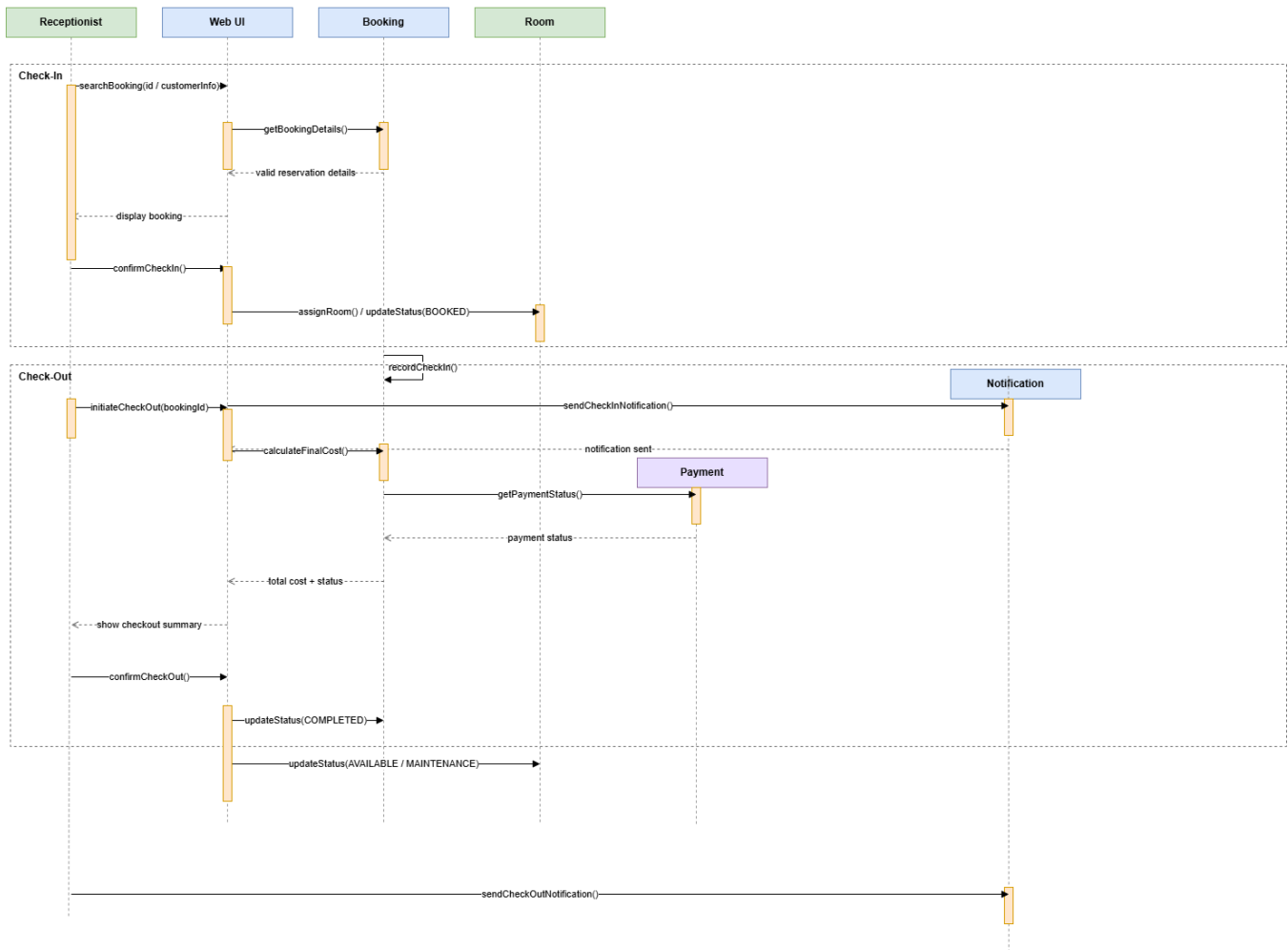


10.3 Sequence Diagram — Online Payment

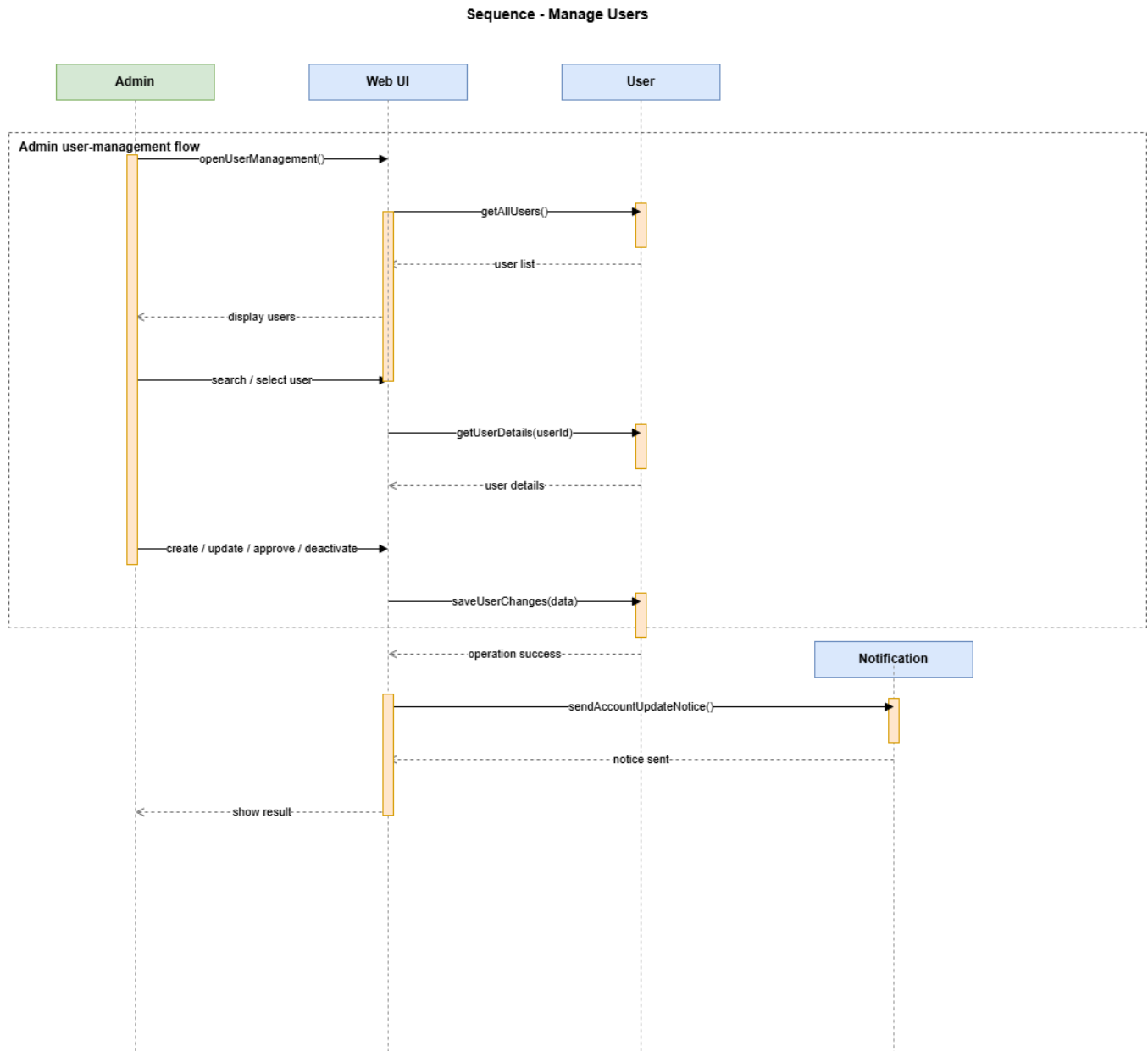


10.4 Sequence Diagram —Confirm-check

Sequence - Confirm Check In / Out

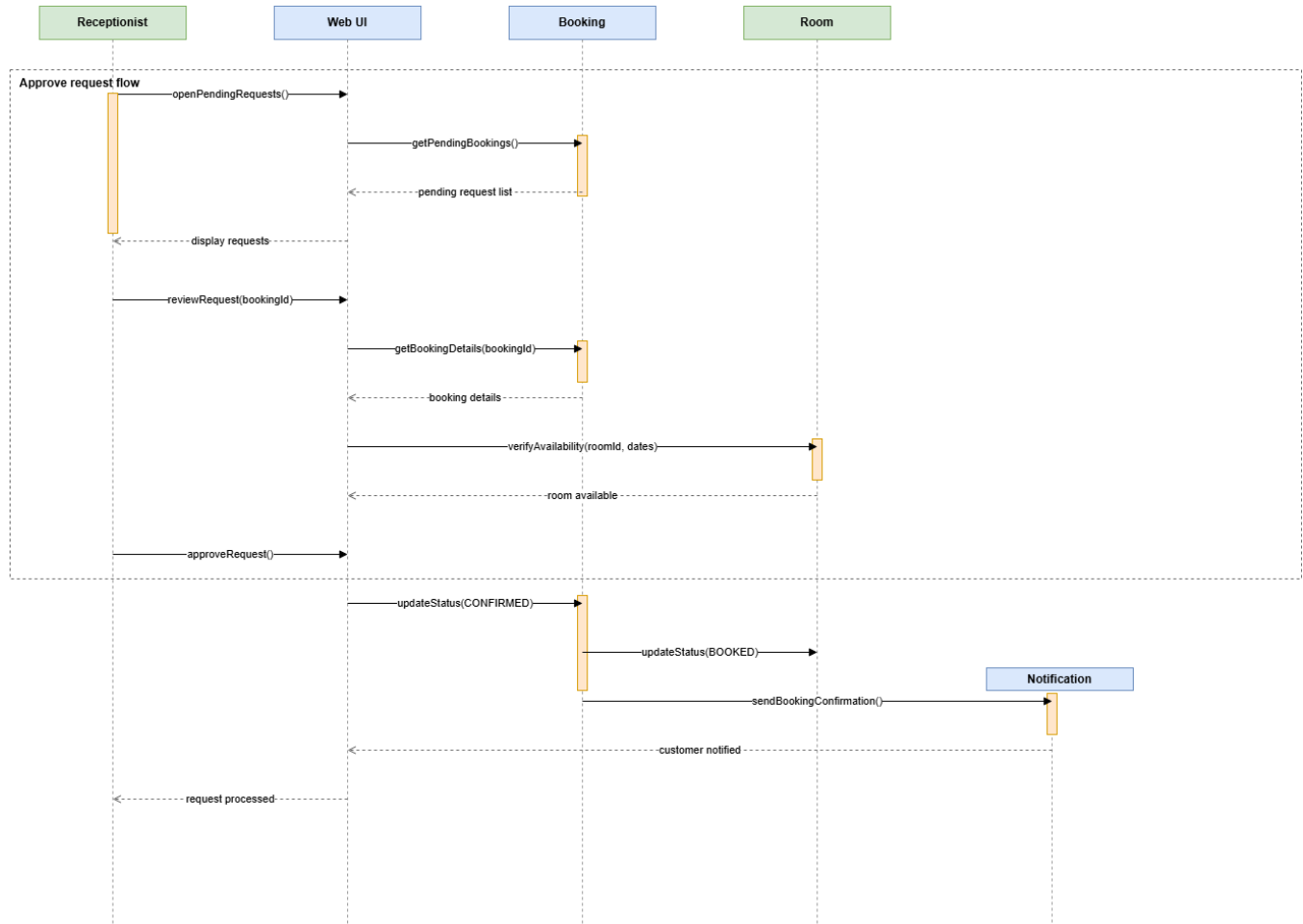


10.5 Sequence Diagram —Manage-Users



10.6 Sequence Diagram —Room- request

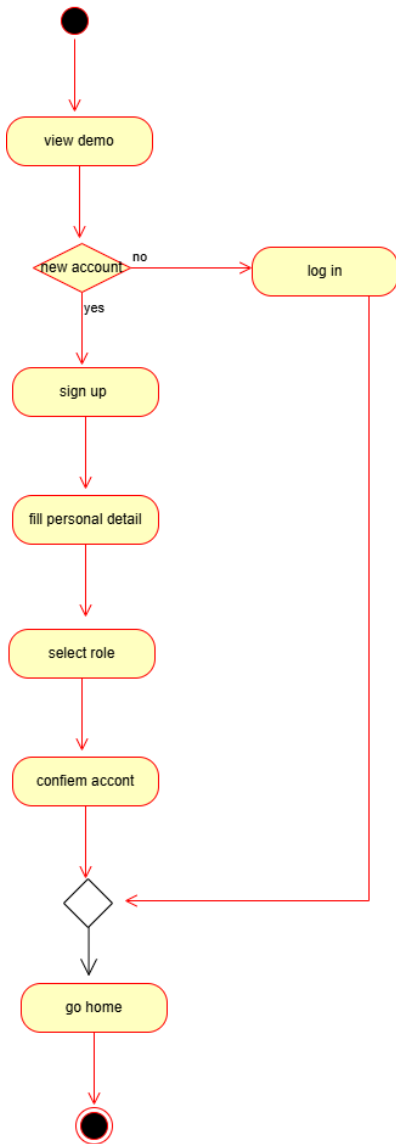
Sequence - Manage Room Request



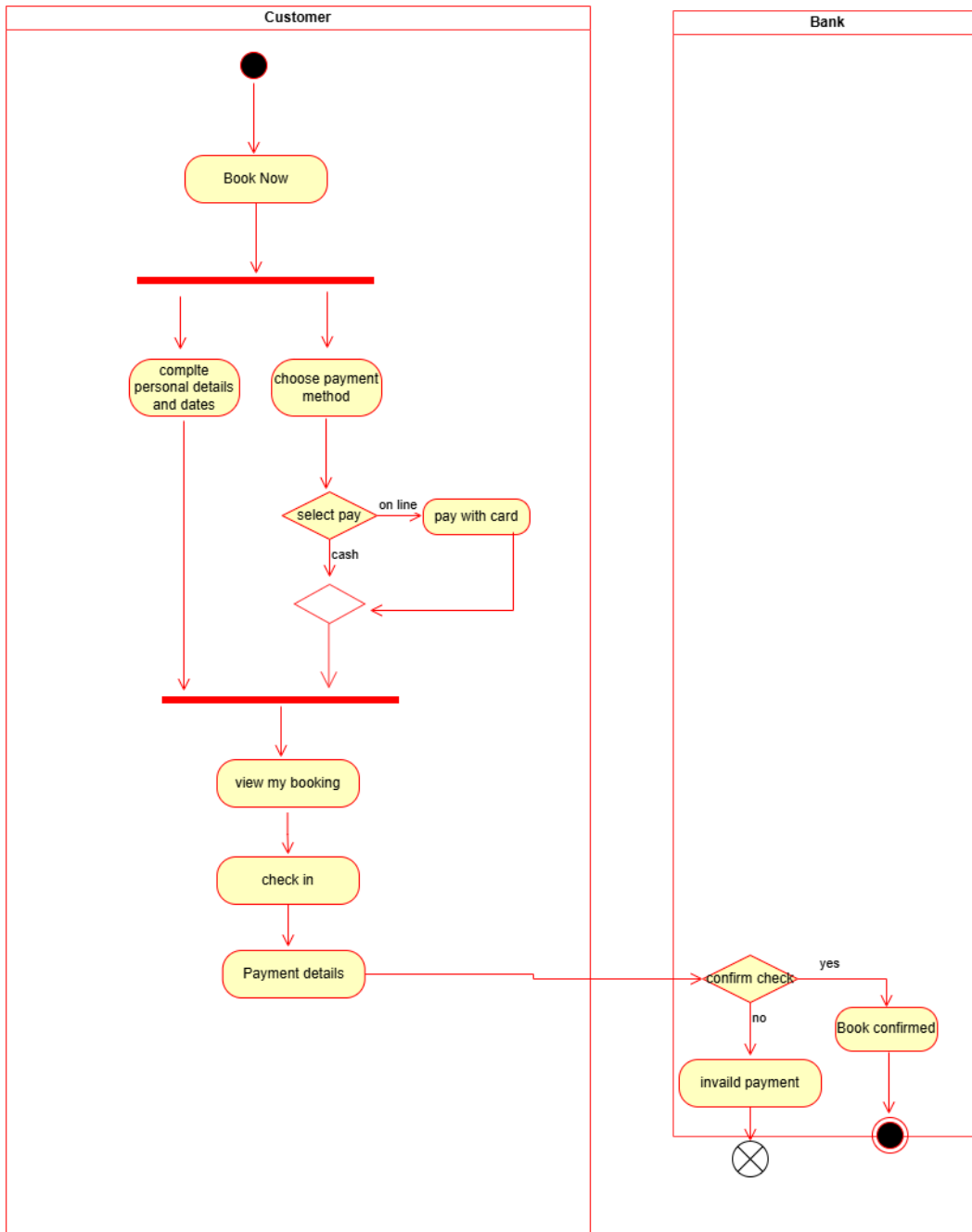
SECTION 11 — ACTIVITY DIAGRAMS

11. Activity Diagrams

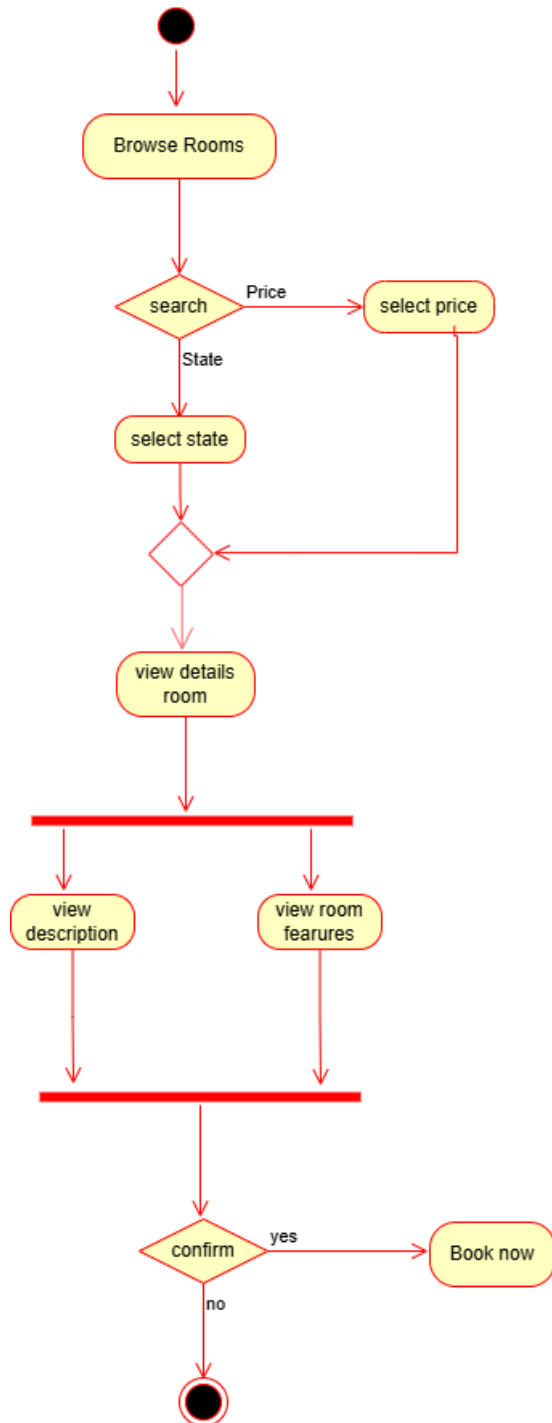
11.1 Activity Diagram — Authentication Flow



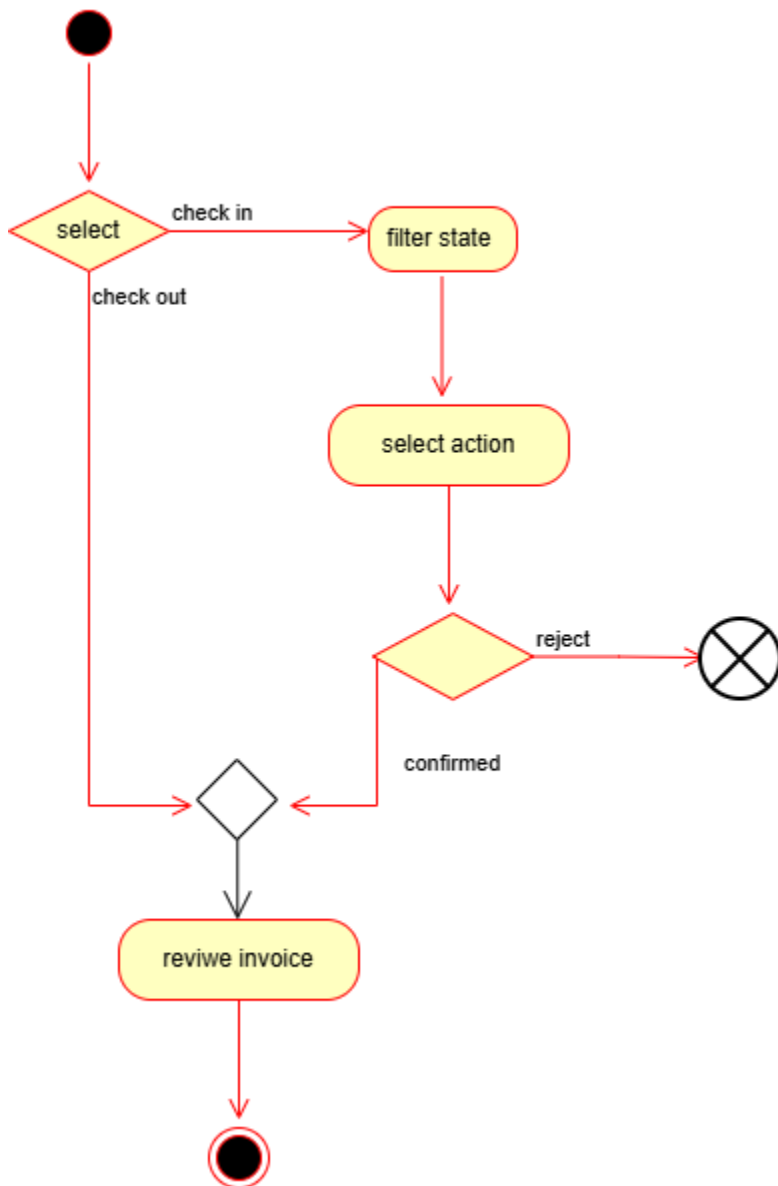
11.2 Activity Diagram — Booking Flow



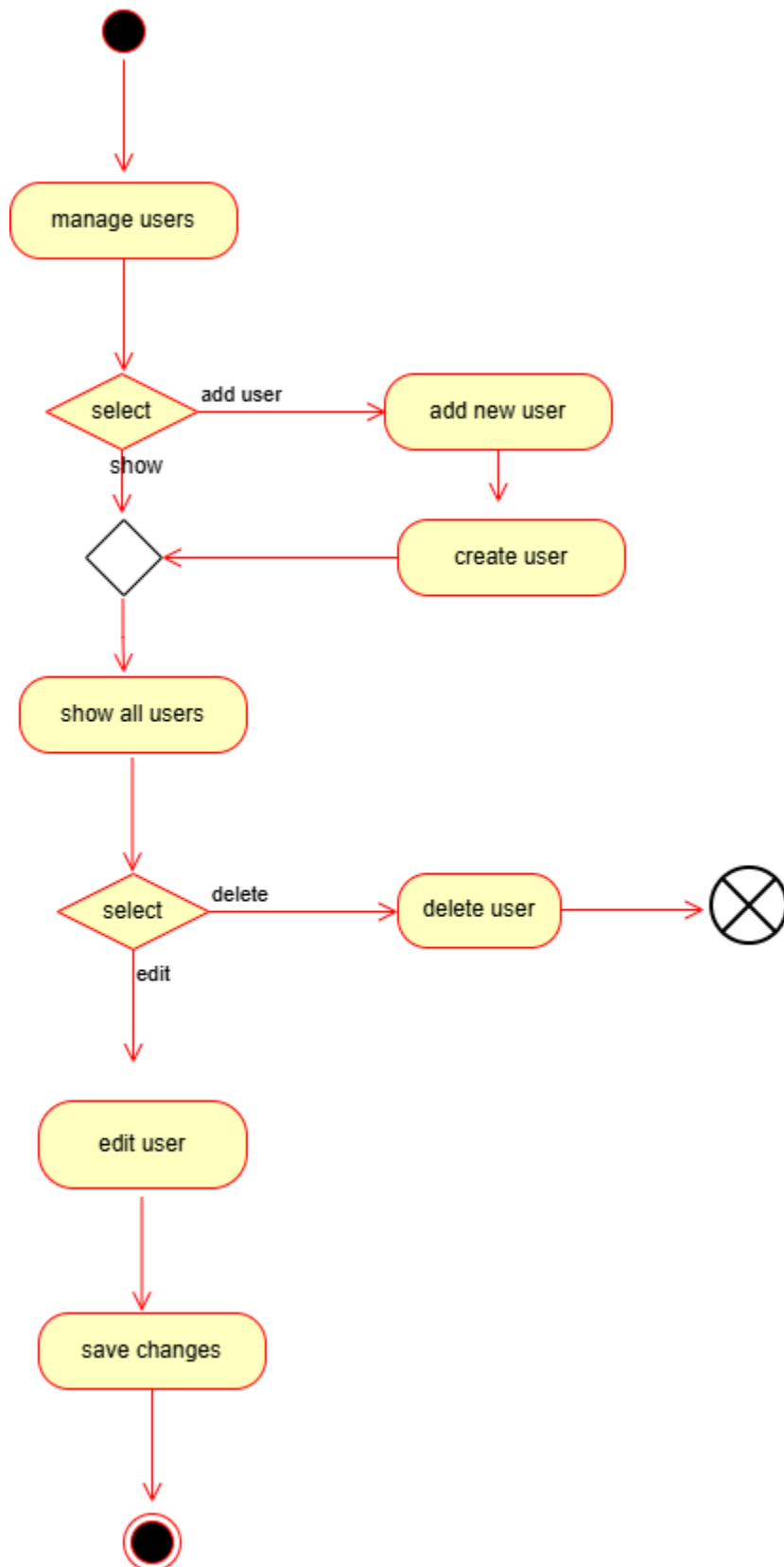
11.3 Activity Diagram — Browse-Room Flow



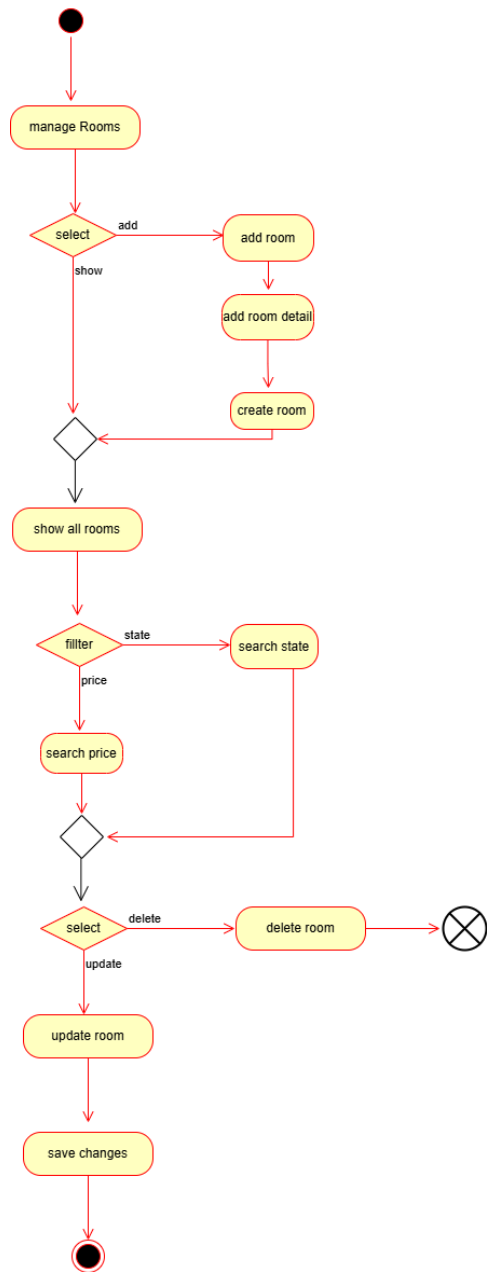
11.4 Activity Diagram — Confirm-Check Flow



11.5 Activity Diagram — Manage-Users Flow



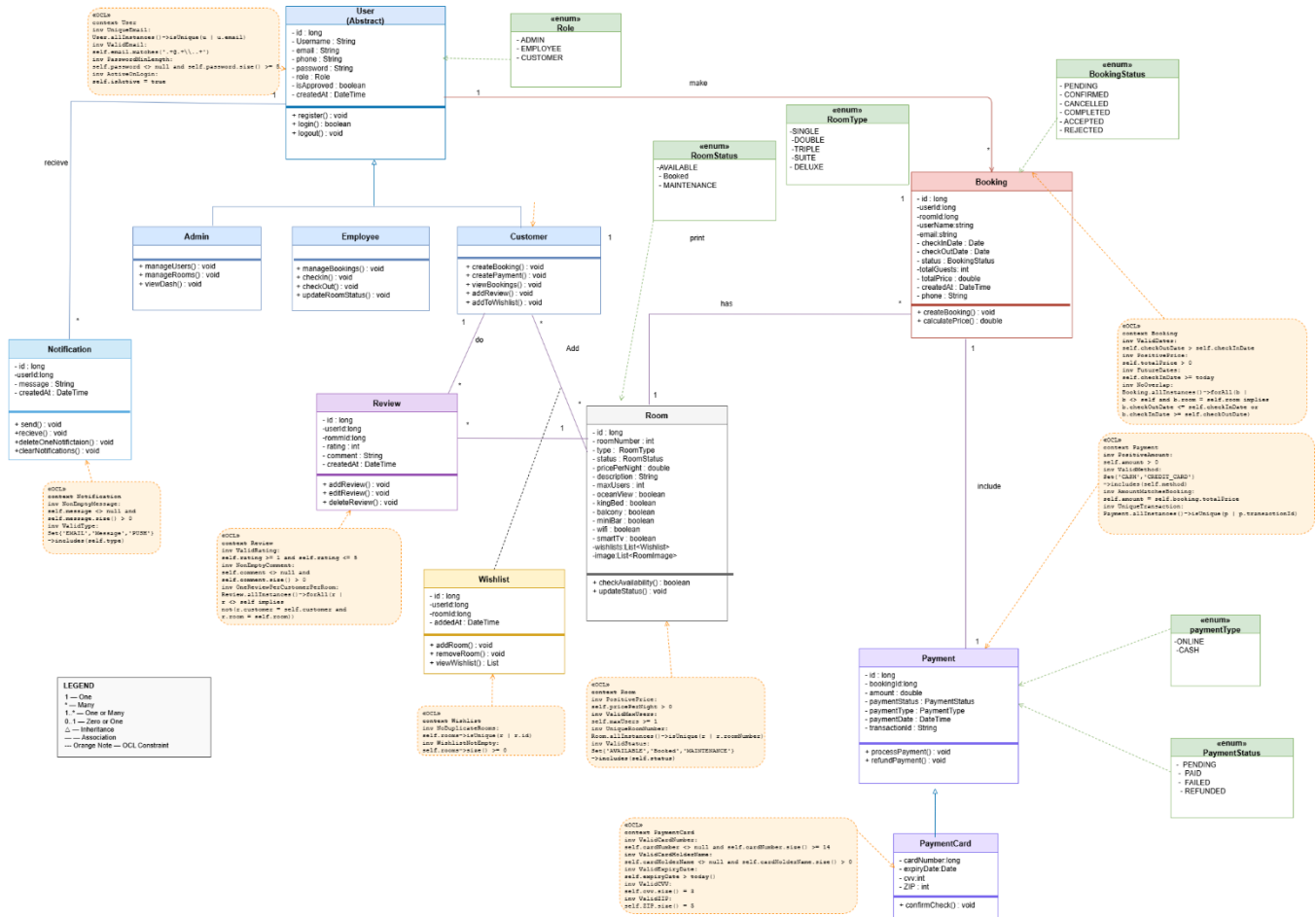
11.6 Activity Diagram — Manage-Room Flow



SECTION 12 — CLASS DIAGRAM

12. Class Diagram

12.1 Domain Class Diagram



12.2 Enumerations

UserRole	RoomStatus	BookingStatus	PaymentMethod
ADMIN	Available	Pending	Cash
RECEPTIONIST	Booked	Confirmed	Online
CUSTOMER	Maintenance	Cancelled	
		Completed	

SECTION 13 — TOOLS & TECHNOLOGIES

13. Tools and Technologies

13.1 Technology Stack

Layer	Technology	Purpose
Backend	Spring Boot 3.x	Microservices framework — REST APIs, business logic
Backend	Spring Security + JWT	Authentication, authorization, and RBAC enforcement
Backend	Spring Data JPA	ORM layer — database access and entity management
Backend	Spring Cloud Gateway	API Gateway — routing, rate limiting, load balancing
Backend	Eureka Server	Service discovery and registration for microservices
Backend	Spring AOP	Aspect-Oriented Programming — logging, auditing
Frontend	React.js	Single-page application — customer and admin UIs
Frontend	Tailwind CSS / MUI	UI styling and responsive design components
Database	MySQL / PostgreSQL	Relational database for all entity persistence
Database	Redis	Caching layer — session management, OTP storage
Payment	Stripe API / PayPal	Secure online payment processing
Notification	SendGrid	Transactional email delivery service
Notification	Twilio	SMS notification and OTP delivery
Containerization	Docker	Containerization of each microservice
Orchestration	Kubernetes	Container orchestration and scaling
Version Control	Git / GitHub	Source code management and collaboration
API Testing	Postman	REST API development and testing
Build Tool	Maven / Gradle	Java project build and dependency management
Cloud	AWS / Azure	Cloud deployment, storage (S3 for images), CDN
Monitoring	ELK Stack	Centralized logging — Elasticsearch, Logstash, Kibana

13.2 UML Modeling Tools

Modeling & Design Tools Used
▶ draw.io (diagrams.net) — for architecture and ER diagrams
▶ Figma — for UI/UX wireframes and prototype designs
▶ Swagger / OpenAPI — for REST API documentation

SECTION 14 — FRONTEND STRUCTURE

14. Frontend Structure

14.1 Customer Pages

Customer Interface Pages
➤ Sign-In / Sign-Up — authentication with OTP verification
➤ Home Page — room listing with filter sidebar (type, price, dates, amenities)
➤ Room Details Page — photos, features, pricing, reviews, "Book Now" / "Add to Wishlist" buttons
➤ Booking Page — date selection, availability check, price summary, confirm button
➤ My Bookings — booking history with status indicators and cancel option
➤ Favorites / Wishlist — saved rooms grid with quick-book access
➤ Checkout Page — final bill summary, payment method selection
➤ Review Page — star rating widget and comment form for completed stays
➤ Profile Page — view/edit personal information, view notifications

14.2 Receptionist Pages

Receptionist Interface Pages
➤ Dashboard — daily overview: pending check-ins, active bookings, recent payments
➤ Booking Requests — list of incoming bookings with accept/reject actions
➤ Room Management — room status grid with availability toggle
➤ Check-In Page — search by booking ID or name, perform check-in, assign room
➤ Check-Out Page — initiate checkout, view final bill, collect payment, generate invoice
➤ Payments Management — list of cash payments and payment statuses

14.3 Admin Pages

Admin Interface Pages
➤ Admin Dashboard — KPI cards: occupancy rate, revenue, active bookings, new registrations
➤ User Management — table of all users with role, VIP/blacklist toggles, search and filter

➤ Room Management — full CRUD for rooms: add, edit, delete, upload photos
➤ Employees Management — manage receptionist accounts and access
➤ Analytics Page — charts for revenue trends, occupancy rates, popular rooms
➤ Logs Page (AOP) — chronological system action log with filtering
➤ Reviews Page — moderation panel for customer reviews
➤ System Configuration — settings for booking rules, payment, and notification preferences

14.4 Shared Components

Shared UI Components
➤ Navigation Bar / Sidebar — role-aware dynamic navigation
➤ Notifications Panel — real-time notification bell with unread count
➤ Profile Dropdown — quick access to profile, settings, and logout
➤ Error Pages — 404 Not Found, 500 Server Error, 403 Forbidden
➤ Loading Spinner / Skeleton Screens — for async data fetching states
➤ Toast Notifications — success, error, and info alerts

SECTION 16 — APPENDIX & GLOSSARY

16. Glossary

Term	Definition
Check-In	The process where a guest arrives at the hotel and is assigned their booked room, initiated by the receptionist.
Check-Out	The process where a guest leaves the hotel, the bill is finalized, and the room is freed.
JWT (JSON Web Token)	A compact, URL-safe token format used for securely transmitting authentication information between parties.
RBAC	Role-Based Access Control — a security approach that restricts system access based on user roles.
OTP	One-Time Password — a temporary code sent via email or SMS for secure account verification.
VIP Customer	A customer classification indicating premium status, potentially offering priority booking or special discounts.
Blacklisted User	A user account that has been suspended and is prevented from making new bookings.
Wishlist	A personal list where customers save rooms they are interested in for future reference.
Booking Status	The current state of a reservation: Pending, Confirmed, Cancelled, or Completed.
Room Status	The current availability state of a hotel room: Available, Booked, or Maintenance.
Invoice	A financial document generated by the system summarizing charges and payment details for a guest stay.
AOP	Aspect-Oriented Programming — used to add cross-cutting concerns (logging, auditing) without modifying core business logic.
Microservices	An architectural style where an application is built as a collection of small, independently deployable services.
API Gateway	A server that acts as a single entry point for all client requests, routing them to appropriate microservices.
Eureka	Netflix Eureka — a service registry used for microservice discovery in Spring Cloud.

SECTION 8 — DESIGN PATTERNS

The following design patterns were identified and applied throughout the Hotel Management System implementation, ensuring clean code, maintainability, and scalability.

8.1 Builder Pattern

Applied in the Room Service when constructing complex Room entity objects. The Builder pattern allows step-by-step construction of Room objects with multiple optional attributes (amenities, features) without requiring a complex constructor.

```
1 public Room addRoom(CreateRoomRequest request) {
2     validateRoomNumberUniqueness(request.getRoomNumber(), null);
3     validateImagesCount(request.getImages());
4
5     Room room = Room.builder()
6         .roomNumber(request.getRoomNumber())
7         .status(RoomStatus.AVAILABLE)
8         .pricePerNight(request.getPricePerNight())
9         .description(request.getDescription())
10        .roomType(request.getRoomType())
11        .maxNumberOfUsers(request.getMaxNumberOfUsers())
12        .oceanView(Boolean.TRUE.equals(request.getOceanView()))
13        .kingBed(Boolean.TRUE.equals(request.getKingBed()))
14        .balcony(Boolean.TRUE.equals(request.getBalcony()))
15        .miniBar(Boolean.TRUE.equals(request.getMiniBar()))
16        .wifi(Boolean.TRUE.equals(request.getWifi()))
17        .smartTv(Boolean.TRUE.equals(request.getSmartTv()))
18        .build();
}
```

Aspect	Details
Pattern	Builder (Creational)
Location	RoomService — addRoom() method
Benefit	Eliminates complex constructors; makes object creation readable and flexible

8.2 Singleton Pattern

The Singleton pattern is applied via Spring's IoC container. Spring Beans are Singletons by default — a single instance of each service/component is created and shared across the application. This is applied to JWT Service, Custom User Details Service, and all service layer components

```

1 package com.bookingService.bookingService.Services;
2
3 import com.bookingService.bookingService.DTO.MonthlyRevenueDTO;
4 import com.bookingService.bookingService.DTO.RoomDTO;
5 import com.bookingService.bookingService.DTO.UserDTO;
6 import com.bookingService.bookingService.DTO.UserStatistics;
7 import com.bookingService.bookingService.Entity.Booking;
8 import com.bookingService.bookingService.Exceptions.BadRequestException;
9 import com.bookingService.bookingService.Exceptions.ResourceNotFoundException;
10 import com.bookingService.bookingService.Repository.BookingRepository;
11 import org.springframework.beans.factory.annotation.Autowired;
12 import org.springframework.stereotype.Service;
13 import java.time.LocalDate;
14 import java.util.HashMap;
15 import java.util.List;
16 import java.util.Map;
17 import java.util.stream.Collectors;
18
19 @Service
20 public class BookingServices {
21
22     @Autowired
23     private BookingRepository repo;
24
25     @Autowired
26     private UserClient userClient;
27
28     @Autowired
29     private RoomClient roomClient;
30
31     @Autowired
32     private PaymentClient paymentClient;
33
34     private void validateBookingId(Long id) {
35         if (id == null || id < 0) {
36             throw new BadRequestException("Invalid booking ID: " + id);
37         }
38     }
39
40     private Booking findById(Long id) {
41         return repo.findById(id)
42             .orElseThrow(() -> new ResourceNotFoundException("Booking not found: " + id));
43     }
44
45     private void validateUserId(Long userId) {
46         UserDTO user = userClient.getUserFromService(userId);
47         if (user == null) {
48             throw new ResourceNotFoundException("User not found with ID: " + userId);
49         }
50     }
51
52     private RoomDTO validateRoomAvailability(Long roomId) {
53         RoomDTO room = roomClient.getRoomFromService(roomId);
54         if (room == null) {
55             throw new ResourceNotFoundException("Room not found with ID: " + roomId);
56         }
57         if (!"AVAILABLE".equals(room.getStatus())) {
58             throw new BadRequestException("Room is not available. Current status: " + room.getStatus());
59         }
60         return room;
61     }
62
63     private void releaseRoom(Long roomId) {
64         roomClient.updateRoomStatus(roomId, "AVAILABLE");
65     }
66
67     private void bookRoom(Long roomId) {
68         roomClient.updateRoomStatus(roomId, "Booked");
69     }
70
71     public Booking createBooking(Booking booking) {
72         validateBookingId(booking.getId());
73         RoomDTO room = validateRoomAvailability(booking.getId());
74
75         double totalPrice = calculateTotalPrice(booking, room);
76         booking.setTotalPrice(totalPrice);
77         booking.setStatus(BookingStatus.PENDING);
78
79         Booking savedBooking = repo.save(booking);
80         bookRoom(booking.getId());
81
82         return savedBooking;
83     }
84
85     public Booking modifyBooking(Long id, Booking booking) {
86         validateBookingId(id);
87         Booking existingBooking = findById(id);
88
89         if (!existingBooking.getId().equals(booking.getId())) {
90             RoomDTO room = validateRoomAvailability(booking.getId());
91             releaseRoom(existingBooking.getId());
92             bookRoom(booking.getId());
93         }
94
95         existingBooking.setCheck_in_date(booking.getCheck_in_date());
96         existingBooking.setCheck_out_date(booking.getCheck_out_date());
97         existingBooking.setRoomId(booking.getRoomId());
98
99         RoomDTO room = roomClient.getRoomFromService(booking.getRoomId());
100         double totalPrice = calculateTotalPrice(existingBooking, room);
101         existingBooking.setTotalPrice(totalPrice);
102     }
103
104     public MonthlyRevenueDTO getMonthlyRevenue(LocalDate startDate, LocalDate endDate) {
105         List<Booking> bookings = repo.findByCheckInDateBetween(startDate, endDate);
106         Map<LocalDate, Double> revenueMap = new HashMap<>();
107         for (Booking booking : bookings) {
108             LocalDate checkInDate = booking.getCheckInDate();
109             Double revenue = revenueMap.get(checkInDate);
110             if (revenue == null) {
111                 revenue = 0.0;
112             }
113             revenue += booking.getTotalPrice();
114             revenueMap.put(checkInDate, revenue);
115         }
116         return new MonthlyRevenueDTO(revenueMap);
117     }
118
119     public UserStatistics getUserStatistics(Long userId) {
120         validateUserId(userId);
121         UserDTO user = userClient.getUserFromService(userId);
122         List<Booking> bookings = repo.findByUserId(userId);
123         Map<String, Integer> statistics = new HashMap<>();
124         for (Booking booking : bookings) {
125             String status = booking.getStatus().name();
126             Integer count = statistics.get(status);
127             if (count == null) {
128                 count = 0;
129             }
130             count++;
131             statistics.put(status, count);
132         }
133         return new UserStatistics(user.getId(), user.getUsername(), statistics);
134     }
135 }

```

Aspect	Details
Pattern	Singleton (Creational)
Location	All Spring @Service, @Component, @Repository beans (JWTService, BookingService, etc.)
Benefit	Ensures a single shared instance; reduces memory overhead; managed lifecycle by Spring container

```

82     return savedBooking;
83 }
84
85 public Booking modifyBooking(Long id, Booking booking) {
86     validateBookingId(id);
87     Booking existingBooking = findBookingOrThrow(id);
88
89     if (!existingBooking.getRoomId().equals(booking.getRoomId())) {
90         RoomID newRoom = validateAndGetAvailableRoom(booking.getRoomId());
91         releaseRoom(existingBooking.getRoomId());
92         bookRoom(booking.getRoomId());
93     }
94
95     existingBooking.setCheck_in_Date(booking.getCheck_in_Date());
96     existingBooking.setCheck_out_Date(booking.getCheck_out_Date());
97     existingBooking.setRoomId(booking.getRoomId());
98
99     RoomID room = roomClient.getRoomFromService(booking.getRoomId());
100     double totalPrice = calculateTotalPrice(existingBooking, room);
101     existingBooking.setTotalPrice(totalPrice);
102
103     return repo.save(existingBooking);
104 }
105
106 public void cancelBooking(Long bookingId) {
107     validateBookingId(bookingId);
108     Booking booking = findBookingOrThrow(bookingId);
109
110     booking.setStatus(BookingStatus.CANCELLED);
111     repo.save(booking);
112     releaseRoom(booking.getRoomId());
113 }
114
115 public List<Booking> viewBookingHistory(Long userId) {
116     validateUserExists(userId);
117     return repo.findByUserId(userId);
118 }
119
120 public List<Booking> viewAllBookings() {
121     return repo.findAll();
122 }
123
124 public int countPendingBookingRequests() {
125     return repo.countByStatus(BookingStatus.PENDING);
126 }
127
128 public int countConfirmedBookings() {
129     return repo.countByStatus(BookingStatus.ACCEPTED);
130 }
131
132 public Booking acceptBooking(Long bookingId) {
133     validateBookingId(bookingId);
134     Booking booking = findBookingOrThrow(bookingId);
135     booking.setStatus(BookingStatus.ACCEPTED);
136     return repo.save(booking);
137 }
138
139 public Double countTotalRevenue() {
140     Double totalRevenue = repo.sumAllRevenue();
141     return totalRevenue != null ? totalRevenue : 0;
142 }
143
144 public void rejectBooking(Long bookingId) {
145     validateBookingId(bookingId);
146     Booking booking = findBookingOrThrow(bookingId);
147
148     String intentId = booking.getPaymentIntentId();
149
150     if (intentId != null && !intentId.isEmpty()) {
151         try {
152             paymentClient.RefundPayment(intentId);
153             System.out.println("Refund processed for booking: " + bookingId);
154         } catch (Exception e) {
155             System.err.println("Refund failed but continuing rejection: " + e.getMessage());
156         }
157     }
158
159     booking.setStatus(BookingStatus.REJECTED);
160     repo.save(booking);
161     releaseRoom(booking.getRoomId());
162 }
163
164 public void checkIn(Long bookingId) {
165     Booking booking = findBookingOrThrow(bookingId);
166
167     if (booking.getStatus() != BookingStatus.ACCEPTED) {
168         throw new BadRequestException("Only accepted bookings can check-in");
169     }
170
171     if (!"PAID".equals(booking.getPaymentStatus())) {
172         throw new BadRequestException("Payment not completed");
173     }
174
175     booking.setStatus(BookingStatus.CHECKED_IN);
176     repo.save(booking);
177 }
178
179 public void checkOut(Long bookingId) {
180     Booking booking = findBookingOrThrow(bookingId);
181     booking.setStatus(BookingStatus.COMPLETED);
182     repo.save(booking);
183     releaseRoom(booking.getRoomId());
184 }
185
186 public void updateStatus(Long bookingId, String status, String paymentIntentId, String paymentStatus) {
187     Booking booking = findBookingOrThrow(bookingId);
188 }

```

```

185 }
186
187 public void updateStatus(Long bookingId, String status, String paymentIntentId, String paymentStatus) {
188     Booking booking = findBookingOrThrow(bookingId);
189     switch (status) {
190         case "ACCEPTED":
191             booking.setStatus(BookingStatus.ACCEPTED);
192             booking.setPaymentIntentId(paymentIntentId);
193             booking.setPaymentStatus(paymentStatus);
194             break;
195         case "CANCELLED":
196             booking.setStatus(BookingStatus.CANCELLED);
197             booking.setPaymentStatus(paymentStatus);
198             releaseRoom(booking.getRoomId());
199             break;
200         default:
201             throw new BadRequestException("Invalid status");
202     }
203     repo.save(booking);
204 }
205
206 private double calculateTotalPrice(Booking booking, RoomDTO room) {
207     long days = ChronoUnit.DAYS.between(booking.getCheck_in_Date(), booking.getCheck_out_Date());
208     if (days == 0) {
209         throw new BadRequestException("Check-out date must be after check-in date");
210     }
211     double total = room.getPricePerNight().doubleValue() * days;
212     if (days > 7) {
213         total = total * 0.9;
214     }
215     return total;
216 }
217
218 public Map<String, Object> getDashboardStats() {
219     Map<String, Object> stats = new HashMap<>();
220     Double totalRevenue = repo.sumAllRevenue();
221     stats.put("totalRevenue", totalRevenue != null ? totalRevenue : 0.0);
222     stats.put("totalBookings", repo.count());
223     stats.put("occupancyRate", calculateOccupancyPercent());
224     return stats;
225 }
226
227 private double calculateOccupancyPercent() {
228     long activeBookings = repo.countCurrentActiveBookings();
229     var totalRooms = roomClient.getTotalRooms();
230     if (totalRooms == null || totalRooms == 0)
231         return 0.0;
232     double rate = ((double) activeBookings / totalRooms) * 100;
233     return Math.round(rate * 10) / 10.0;
234 }
235
236 public List<MonthlyRevenueDTO> getMonthlyRevenue() {
237     List<Object> results = repo.findMonthlyRevenue();
238     return results.stream()
239         .map(row -> new MonthlyRevenueDTO(
240             ((Number) row[0]).intValue(), // month
241             ((Number) row[1]).intValue(), // year
242             ((Number) row[2]).doubleValue() // totalRevenue
243         ))
244         .collect(Collectors.toList());
245 }
246
247 public List<Booking> getBookingsByUserIdAndStatus(Long userId, String status) {
248     validateUserExists(userId);
249     try {
250         BookingStatus bookingStatus = BookingStatus.valueOf(status.toUpperCase());
251         return repo.findByUserIdAndStatus(userId, bookingStatus);
252     } catch (IllegalArgumentException e) {
253         throw new BadRequestException("Invalid booking status: " + status);
254     }
255 }
256
257 public boolean canReview(Long userId, Long roomId) {
258     validateUserExists(userId);
259     List<BookingStatus> allowedStatuses = List.of(
260         BookingStatus.CHECKED_IN,
261         BookingStatus.COMPLETED
262     );
263     return repo.existsByUserIdAndRoomIdAndStatusIn(
264         userId,
265         roomId,
266         allowedStatuses
267     );
268 }
269
270 public Booking getPendingBookingForUser(Long userId, Long roomId) {
271     List<Booking> bookings = repo.findByUserIdAndStatus(userId, BookingStatus.PENDING);
272     return bookings.stream()
273         .filter(b -> b.getRoomId() == roomId)
274         .findFirst()
275         .orElse(null);
276 }
277 }

```

8.3 Reverse Proxy Pattern (API Gateway)

The API Gateway acts as a Reverse Proxy — it hides the internal microservice architecture from clients. The client communicates only with port 8080 (the gateway), never knowing the existence of USER-SERVICE (8081), ROOM-SERVICE (8082), etc. This provides security, abstraction, and a single entry point.

Aspect	Details
Pattern	Reverse Proxy / API Gateway (Structural)
Location	APIGateway service — Spring Cloud Gateway
Benefit	Hides internal services; enables centralized JWT validation, rate limiting, and routing
Example	Client calls <code>http://gateway:8080/rooms</code> → Gateway routes to <code>http://ROOM-SERVICE:8082/rooms</code> via Eureka discovery